NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Information Technology

COMPLEX COMPUTING PROBLEM (CCP)

SMART HOME AUTOMATION SYSTEM

Applying OOP Principles to Real-World IoT Computing

GROUP LEADER	SYEDA FAIZA ASLAM [CT-25064]
GROUP MEMBER	MAHREEN [CT-25069]
SECTION	B
COURSE	OBJECT ORIENTED PROGRAMMING
COURSE CODE	CT-260
SEMESTER	SPRING
INSTRUCTOR	MUHAMMAD AHMED ZAKI
SUBMISSION	WEEK 15

TABLE OF CONTENTS

1. PROBLEM STATEMENT

1.1 Background & Context

In the modern world, technology is increasingly being integrated into everyday living environments. A Smart Home Automation System is a real-world computing application that enables centralized, intelligent control of household devices such as lights, fans, air conditioners, security cameras, and door locks. Instead of manually operating each device individually, a smart home allows devices to be controlled remotely through a software interface and to respond automatically to environmental conditions detected by sensors.

The significance of such a system lies in three key areas: convenience, energy efficiency, and security. Residents can control all their home devices from a single point, reduce electricity wastage through automated shutdowns, and enhance home security through automatic locking and camera surveillance.

1.2 The Computing Challenge

The core computing challenge in designing a Smart Home Automation System is managing a diverse collection of device types in a unified, scalable, and maintainable way. Specifically, the system must:

- Support multiple device types, each with its own unique attributes and behaviours.
- Provide a consistent control interface regardless of which device is being operated.
- Allow new device types to be added in the future without restructuring existing code.
- Enable event-driven automation where devices respond automatically to sensor signals.
- Support manual user control alongside automated responses.
- Maintain a command history that allows actions to be undone.

These challenges make this problem an ideal candidate for an Object-Oriented Programming solution. OOP principles — Encapsulation, Inheritance, Polymorphism, and Abstraction — directly address each of these requirements by organising the system into a clean, modular class hierarchy with well-defined responsibilities.

1.3 OOP Relevance

This problem maps naturally to OOP because each physical device in a smart home is a real-world **object** with its own **data (attributes)** and **behaviour (methods)**. Grouping devices into a class hierarchy allows shared properties to be defined once and reused across all device types, while device-specific behaviour is handled through method overriding. The event-driven nature of the system further aligns with the Observer design pattern, a cornerstone of modern OOP-based system design.

2. OOP PRINCIPLES IDENTIFICATION

All four OOP pillars defined in the CT-260 course plan are applicable to this system. Each principle is described below with clear justification and concrete examples from the implementation.

2.1 Encapsulation

Definition:

Encapsulation is the bundling of data and the methods that operate on that data within a single class, while restricting direct external access to internal data using access modifiers.

In the Smart Home System, every class encapsulates its own data using private and protected access modifiers. For example:

- The SmartLight class contains private attributes brightness (int, 0–100%) and colorMode (string). These cannot be accessed or modified directly from outside the class. External code must use the public methods setBrightness() and setColor() to interact with them.
- The SmartLock class encapsulates a private accessCode (string). No part of the system can read or change this code directly — it is only used internally during the unlock() operation to validate user input.
- The HomeController class encapsulates its device list (vector<Device*>) and command history (vector<ICommand*>), exposing only clean public methods like addDevice(), executeCommand(), and broadcastEvent().

Justification:

Encapsulation prevents unintended data modification, keeps each class self-contained, and makes the system safer and easier to maintain.

2.2 Inheritance

Definition:

Inheritance is a mechanism where a derived class acquires the properties and behaviours of a base class, establishing an is-a relationship and enabling code reuse.

The system uses a single abstract base class Device from which five concrete device classes inherit:

Derived Class	Base Class	Relationship	Added Attributes
SmartLight	Device	is-a Device	brightness, colorMode
SmartFan	Device	is-a Device	speed (1–5)
AirConditioner	Device	is-a Device	targetTemp, mode
SecurityCamera	Device	is-a Device	isRecording
SmartLock	Device	is-a Device	isLocked, accessCode

The Device base class defines shared attributes (deviceName, location, isOn, powerWatts) and declares pure virtual methods (turnOn(), turnOff(), getStatus(), getType()). Each derived class inherits these shared properties and provides its own implementation of the virtual methods.

Justification:

Inheritance eliminates code duplication. Common device properties are written once in the Device base class and inherited by all five device classes, making the codebase compact and easier to extend.

2.3 Polymorphism

Definition:

Polymorphism allows objects of different types to be treated as objects of a common base type, with the actual behaviour determined at runtime based on the object's true type.

In the system, all device objects are stored in a vector<Device*> — a list of base class pointers. When turnOn(), turnOff(), or getStatus() is called on any pointer in this list, C++ virtual dispatch automatically executes the correct overridden version for each device type:

- Calling turnOn() on a SmartLight pointer prints brightness and color.
- Calling turnOn() on a SmartFan pointer prints fan speed.
- Calling turnOn() on an AirConditioner pointer prints mode and temperature.

Similarly, the onNotify() method is polymorphic — each device implements its own response to the same broadcast event. When a HIGH_TEMP event is broadcast, the SmartFan increases its speed to maximum while the AirConditioner sets its temperature to 20°C.

Justification:

Polymorphism allows the HomeController to manage all devices through a single unified interface, regardless of their specific type. This makes the system highly flexible and extensible.

2.4 Abstraction

Definition:

Abstraction hides the internal implementation details of a class and exposes only the essential interface needed for interaction.

The Device class is declared abstract by including pure virtual methods (= 0). This means the Device class cannot be instantiated directly — it only defines what every device must do, not how it does it. This is abstraction: the system works with the concept of a Device without needing to know whether it is a light, a fan, or a lock.

The IObserver and ICommand interfaces further embody abstraction. The HomeController interacts with devices through the IObserver interface (calling onNotify()) and executes operations through the ICommand interface (calling execute() and undo()), without any knowledge of the specific device or operation type involved.

Justification:

Abstraction reduces system complexity by hiding implementation details, allowing each component to evolve independently without affecting the rest of the system.

3. SYSTEM DESIGN & UML

3.1 System Architecture Overview

The Smart Home Automation System is structured into four logical layers, each with clear responsibilities:

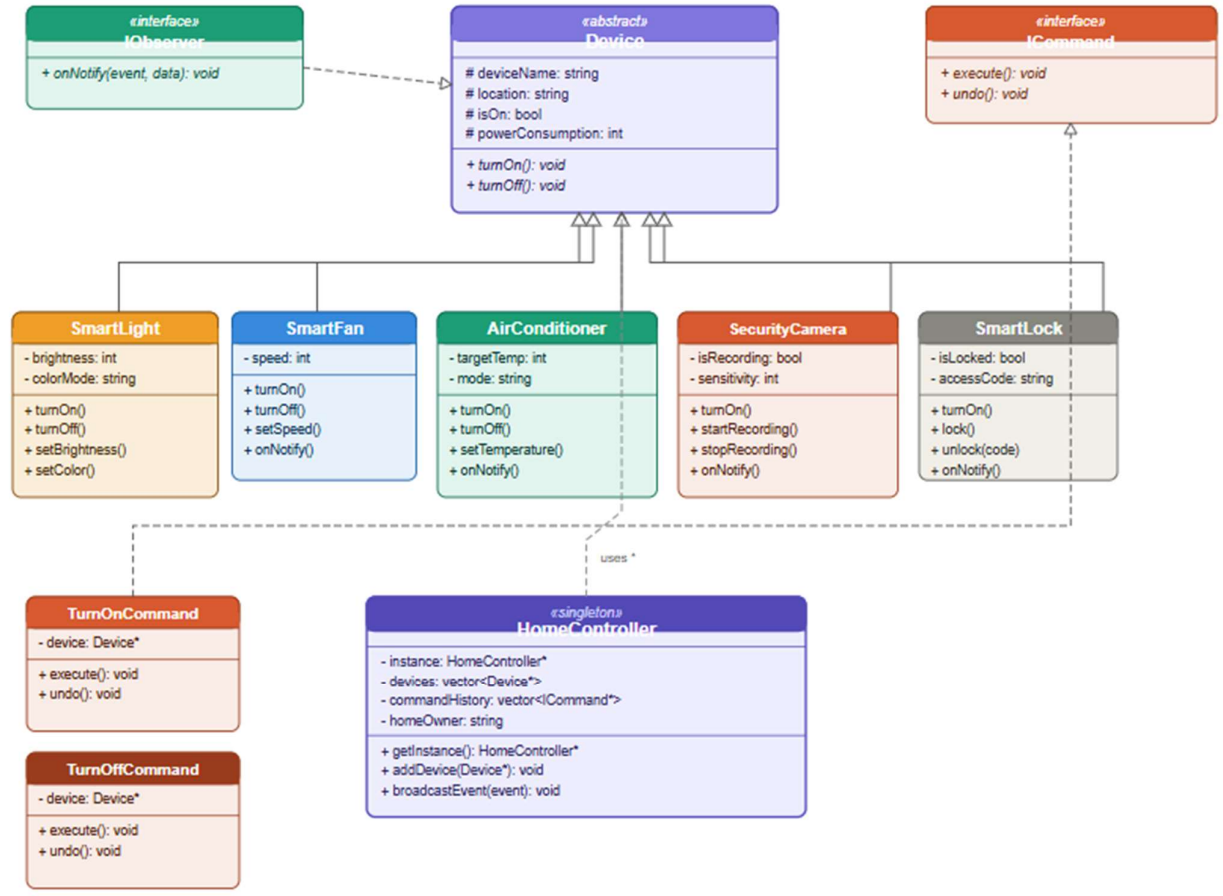
- Device Layer: Five concrete device classes (SmartLight, SmartFan, AirConditioner, SecurityCamera, SmartLock) inheriting from the abstract Device base class.
- Interface Layer: Two interfaces — IObserver (for event notification) and ICommand (for command encapsulation with undo support).
- Control Layer: The singleton HomeController that manages all devices, executes commands, and broadcasts events.
- Interaction Layer: The menu-driven console interface through which users control devices and simulate sensor events.

3.2 Class Responsibilities

Class	Responsibility
Device (Abstract)	Base class defining the common interface and shared attributes for all devices.
SmartLight	Controls lighting with adjustable brightness (0–100%) and colour mode.
SmartFan	Controls fan operation with adjustable speed settings (1–5).
AirConditioner	Controls cooling/heating with configurable temperature (16–30°C) and mode.
SecurityCamera	Manages camera activation and video recording for security monitoring.
SmartLock	Manages door locking and unlocking via a secure access code.
IObserver	Interface requiring onNotify() — enables event-driven device behaviour.
ICommand	Interface requiring execute() and undo() — enables command history and undo.
TurnOnCommand	Encapsulates the Turn ON action with corresponding undo (Turn OFF).
TurnOffCommand	Encapsulates the Turn OFF action with corresponding undo (Turn ON).
HomeController	Singleton central hub managing devices, events, commands, and status reports.

3.3 UML Class Diagram

Figure 1 below illustrates the complete UML class diagram for the Smart Home Automation System. The diagram uses standard UML notation including solid arrows for inheritance relationships, dashed arrows for interface realization, and aggregation for the HomeController's device collection.



The diagram clearly shows the inheritance hierarchy from Device to all five concrete classes, the realization of IObserver by the Device class, the realization of ICommand by TurnOnCommand and TurnOffCommand, and the aggregation relationship between HomeController and Device.

3.4 Key Relationships

Relationship	Type	Description
SmartLight / Fan / AC / Camera / Lock → Device	Inheritance	All device classes inherit shared attributes and interface from Device.
Device → IObserver	Realization	Device implements the observer interface for event notifications.
TurnOnCommand / TurnOffCommand → ICommand	Realization	Command classes implement execute() and undo() operations.
HomeController → Device*	Aggregation	Controller holds a collection of Device pointers for unified management.
HomeController → ICommand*	Aggregation	Controller maintains a command history stack for undo functionality.

4. SOLUTION DEFINITION

The Smart Home Automation System is implemented in C++ and operates as a console-based interactive application. The solution is built around two complementary modes of operation: manual user control (simulating a mobile app interface) and automated event-driven responses (simulating real-world IoT sensors).

4.1 System Modules

Module	Menu Option	What It Does
Device Status	Option 1	Displays real-time status of all 6 registered devices including location, on/off state, and power consumption.
Manual Control	Option 2	User selects any device and performs actions: ON/OFF, adjust brightness, set fan speed, change temperature, lock/unlock, start/stop recording.
Auto Events	Option 3	Simulates 8 sensor events that trigger automatic device responses across the entire system.
Undo Command	Option 4	Reverses the most recently executed manual command using the Command Pattern history stack.
Power Report	Option 5	Calculates and displays total active power consumption of all currently ON devices.

4.2 Event-Response Mapping

The following table summarises how each sensor event triggers automatic responses across devices:

Sensor Event	Devices Affected	Automatic Response
MOTION_DETECTED	SmartLight (same room)	Light automatically turns ON in the detected room.
NO_MOTION	SmartLight (same room)	Light automatically turns OFF when no motion is detected.
HIGH_TEMP	SmartFan, AirConditioner	Fan speed set to maximum (5/5); AC turns ON at 20°C.
NORMAL_TEMP	SmartFan	Fan speed reduced to low (2/5) when temperature normalises.
INTRUDER_ALERT	SecurityCamera, SmartLock	Camera activates and begins recording; Lock auto-locks immediately.

LEAVING_HOME	SmartFan, AC, SmartLock	Fan and AC turn OFF; Lock auto-locks to secure the home.
DAYLIGHT	SmartLight	All lights turn OFF automatically when daylight is detected.

4.3 Step-by-Step OOP Solution

1. **Device Instantiation:**
   Five concrete device objects are created, each with a name, location, and power rating. Each object encapsulates its own private state.
2. **Device Registration:**
   All device objects are registered with the singleton HomeController using addDevice(). The controller stores them as Device* pointers enabling polymorphic access.
3. **Manual Control:**
   User selects a device and action from the menu. The action is wrapped in a TurnOnCommand or TurnOffCommand object, executed via the controller, and saved to command history for undo support.
4. **Event Broadcasting:**
   When a sensor event is triggered, HomeController::broadcastEvent() iterates all registered devices and calls onNotify() on each. Each device independently decides whether and how to respond.
5. **Undo Operation:**
   The last command is retrieved from the history stack, its undo() method is called (which executes the inverse operation), and the command is removed from history.
6. **Reporting:**
   displayStatus() iterates all devices polymorphically calling displayInfo() on each. powerReport() sums the wattage of all active devices.

4.4 Pseudocode

```
// Singleton: get one central controller
HomeController* home = HomeController::getInstance();
// Create and register devices
home->addDevice(new SmartLight("Living Room Light", "Living
Room"));
// Command Pattern: execute with undo support
home->executeCommand(new TurnOnCommand(lightPtr));
// Observer Pattern: broadcast to all devices
home->broadcastEvent("HIGH_TEMP");
// Undo last action
home->undoLastCommand();
```

5. DESIGN PATTERN APPLICATION

Three well-established Gang of Four (GoF) design patterns were applied in this system. Each pattern was selected based on a specific design need and directly improves the quality, flexibility, and maintainability of the solution.

5.1 Singleton Pattern

Aspect	Details
Applied To	HomeController class
Problem	A smart home must have exactly one central control hub. If multiple HomeController objects existed, devices could be registered twice and events broadcast inconsistently.
Solution	The constructor is made private. A static getInstance() method creates the object on first call and returns the same instance on all subsequent calls, guaranteeing only one controller exists.
Benefit	Eliminates inconsistent state caused by multiple controller instances. Ensures all devices are managed by a single, unified hub.

5.2 Observer Pattern

Aspect	Details
Applied To	HomeController (subject) and all Device subclasses (observers)
Problem	When a sensor detects an event, multiple devices need to respond automatically. Hardcoding responses in the controller would create tight coupling and make the system rigid.
Solution	All devices implement the IObserver interface with an onNotify() method. When HomeController::broadcastEvent() is called, it notifies all registered devices, each of which independently decides how to respond.
Benefit	Fully decoupled event handling. New devices can be added and will automatically participate in event-driven automation without any changes to the controller.

5.3 Command Pattern

Aspect	Details
Applied To	TurnOnCommand and TurnOffCommand classes
Problem	Device operations need to be stored, executed, and reversed. Without encapsulation, implementing undo would require storing raw state snapshots.
Solution	Each device operation is wrapped in a command object implementing the ICommand interface with execute() and undo() methods. Executed commands are stored in a history stack in the HomeController.
Benefit	Supports undo functionality cleanly. The controller is completely decoupled from device-specific operations. New command types can be added without modifying the controller, following the Open/Closed Principle.

6. LIMITATIONS & FUTURE WORK

6.1 Current Limitations

- Hardware Simulation Only: The system is entirely software-based and does not connect to real physical devices or IoT hardware. Real sensors, actuators, and communication protocols (such as MQTT or Zigbee) are not implemented.
- No Data Persistence: All device states and command history exist in memory only. When the program exits, all data is lost. A production system would require database integration or file-based state saving.
- Console-Based Interface: The system uses a text menu rather than a graphical interface. While fully functional for demonstrating OOP concepts, a real smart home system would use a mobile or web-based GUI.
- Single User Model: The current design supports only one user with a fixed access code (1234). A real system would require multi-user authentication with role-based permissions.
- Fixed Device Set: Devices are hardcoded in the main function. A production system would allow dynamic device registration and removal at runtime.
- No Real-Time Scheduling: The system does not support timed automation (e.g., turn on lights at 7:00 AM). A scheduler or timer thread would be required.

6.2 Future Enhancements

- Real Hardware Integration: Connect the system to physical IoT devices using protocols such as MQTT, Zigbee, or Wi-Fi APIs from platforms like Home Assistant or AWS IoT Core.
- Graphical User Interface: Build a mobile app or web dashboard using Qt (C++) or a web framework to provide a visual, user-friendly control interface.
- Factory Pattern: Introduce a DeviceFactory class to dynamically create device objects based on user input or configuration files, making the system more flexible.
- Strategy Pattern: Allow users to define custom automation rules and strategies for how devices should respond to different events, making the system configurable.
- Data Persistence: Use file I/O or a lightweight database (SQLite) to save and restore device states between sessions.
- Energy Analytics: Track historical power consumption data and provide reports, recommendations, and alerts when energy usage exceeds defined thresholds.
- Voice Control Integration: Integrate with voice assistants such as Google Assistant or Amazon Alexa to enable hands-free device control.

7. TEAM CONTRIBUTION TABLE

The following table outlines the specific contributions made by each group member to the development of this project. The work was divided based on individual strengths and the core components of the CCP.

Member Name	Roll No.	Role	Specific Contributions
SYEDA FAIZA ASLAM	CT-25064	Group Leader & Core Developer	• Designed overall system architecture and class hierarchy • Implemented all C++ source code including Device base class, all five device subclasses, HomeController Singleton, and Command Pattern classes • Implemented interactive menu system for user control • Wrote Report Sections: 1 (Problem Statement), 3 (System Design), 4 (Solution Definition), 5 (Design Patterns) • Managed overall project coordination and submission
MAHREEN	CT-25069	UML Designer & Report Author	• Designed and drew the complete UML class diagram using draw.io, applying standard UML notation for all relationships • Conducted research on OOP principles and design patterns relevant to smart home systems • Tested C++ code in Dev C++ and reported bugs for correction • Wrote Report Sections: 2 (OOP Principles), 6 (Limitations & Future Work), 7 (Team Contribution) • Handled report formatting, proofreading, and final review

REFERENCES

The following sources were consulted during the research and development of this project. References are listed in a consistent citation format.

[1] B. Stroustrup, *The C++ Programming Language,* 4th ed. Boston, MA: Addison-Wesley, 2013.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software.* Reading, MA: Addison-Wesley, 1994.

[3] G. Booch, R. A. Maksimchuk, and M. W. Engle, *Object-Oriented Analysis and Design with Applications,* 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2007.

[4] J. Rumbaugh, I. Jacobson, and G. Booch, *The Unified Modeling Language Reference Manual,* 2nd ed. Boston, MA: Addison-Wesley, 2004.

[5] CT-260 Course Plan — Object Oriented Programming, Department of Computer Science & Information Technology, NED University of Engineering & Technology, Spring 2026.

[6] S. B. Lippman, J. Lajoie, and B. E. Moo, *C++ Primer,* 5th ed. Upper Saddle River, NJ: Addison-Wesley, 2012.